

Chapter 7: Input/Output and Command-line Processing

The main agenda of this chapter is to:

- Learn the shell's mechanisms for doing file-oriented input and output.
- Learn about the I/O at the line and word level, which involves moving information between the domains of files/terminals and shell variables.

I/O Redirectors: Redirection in Bash allows a command to **take input from a file or another source**, or **send output somewhere other than the screen**.

Redirector	Function
<code>> file</code>	Force standard output to <i>file</i> even if noclobber is set.
<code>n> file</code>	Force output to <i>file</i> from file descriptor <i>n</i> even if noclobber is set.
<code><> file</code>	Use <i>file</i> as both standard input and standard output.
<code>n<> file</code>	Use <i>file</i> as both input and output for file descriptor <i>n</i> .
<code><< label</code>	Here-document; see text.
<code>n > file</code>	Direct file descriptor <i>n</i> to <i>file</i> .
<code>n < file</code>	Take file descriptor <i>n</i> from <i>file</i> .

Redirector	Function
<code>cmd1 cmd2</code>	Pipe; take standard output of <i>cmd1</i> as standard input to <i>cmd2</i> .
<code>> file</code>	Direct standard output to <i>file</i> .
<code>< file</code>	Take standard input from <i>file</i> .
<code>>> file</code>	Direct standard output to <i>file</i> ; append to <i>file</i> if it already exists.

Redirector	Function
<code>n>> file</code>	Direct file descriptor <i>n</i> to <i>file</i> ; append to <i>file</i> if it already exists.
<code>n>&</code>	Duplicate standard output to file descriptor <i>n</i> .
<code>n<&</code>	Duplicate standard input from file descriptor <i>n</i> .
<code>n>&m</code>	File descriptor <i>n</i> is made to be a copy of the output file descriptor.
<code>n<&m</code>	File descriptor <i>n</i> is made to be a copy of the input file descriptor.
<code>&>file</code>	Directs standard output and standard error to <i>file</i> .
<code><&-</code>	Close the standard input.

Redirector	Function
<code>>&-</code>	Close the standard output.
<code>n>&-</code>	Close the output from file descriptor <i>n</i> .
<code>n<&-</code>	Close the input from file descriptor <i>n</i> .
<code>n>&word</code>	If <i>n</i> is not specified, the standard output (file descriptor 1) is used. If the digits in word do not specify a file descriptor open for output, a redirection error occurs. As a special case, if <i>n</i> is omitted, and word does not expand to one or more digits, the standard output and standard error are redirected as described previously.
<code>n<&word</code>	If word expands to one or more digits, the file descriptor denoted by <i>n</i> is made to be a copy of that file descriptor. If the digits in word do not specify a file descriptor open for input, a redirection error occurs. If word evaluates to -, file descriptor <i>n</i> is

Redirector	Function
	closed. If <i>n</i> is not specified, the standard input (file descriptor 0) is used.
<code>n>&digit-</code>	Moves the file descriptor digit to file descriptor <i>n</i> , or the standard output (file descriptor 1) if <i>n</i> is not specified.
<code>n<&digit-</code>	Moves the file descriptor digit to file descriptor <i>n</i> , or the standard input (file descriptor 0) if <i>n</i> is not specified. digit is closed after being duplicated to <i>n</i> .

Here-document:

Let you feed multiline(Input on different lines) into a command

```
command <<END
line 1
line 2
END
```

The syntax is: (The trick: << pointy tip to the input)

Everything between <<END and END is passed as input to the command.

(If you type <<END, the command ends with end, if typed <<EXIT, it ends with EXIT, so it's an end marker, and you can decide what must be the end marker, also no space between << and the end marker)

```
● gayat@GSP-HP:~$ cat <<END
> hello
> testing
> okay, bye
> END
hello
testing
okay, bye
○ gayat@GSP-HP:~$

● gayat@GSP-HP:~$ cat <<EXIT
> test
> text
> hehe
> EXIT
test
text
hehe
```

```
● gayat@GSP-HP:~$ wc -l <<EXIT
one
two
three
four
EXIT
4
```

Wc→ word count; -l→ counts lines, -w→ counts words, wc alone, gives the count of lines, words and even characters/bytes

```
● gayat@GSP-HP:~$ wc <<EXIT
one
two
three
four
EXIT
4      4      19
```

one	→ 1 word, 3 chars + 1 newline
two	→ 1 word, 3 chars + 1 newline
three	→ 1 word, 5 chars + 1 newline
four	→ 1 word, 4 chars + 1 newline

```
● gayat@GSP-HP:~$ TEMP=bugsreport.txt
● gayat@GSP-HP:~$ echo $TEMP
bugsreport.txt
● gayat@GSP-HP:~$ USER="Gayathri"
● gayat@GSP-HP:~$ BUGADDAR="bugs@example.com"
● gayat@GSP-HP:~$ MACHINE="HP-GSP"
● gayat@GSP-HP:~$ echo OS_VERSION
OS_VERSION
● gayat@GSP-HP:~$ echo "OS_VERSION"
OS_VERSION
● gayat@GSP-HP:~$ OS="Ubuntu 22.04"
● gayat@GSP-HP:~$ CC="gcc"
● gayat@GSP-HP:~$ CFLAGS="-Wall -O2"
● gayat@GSP-HP:~$ cat > $TEMP <<EOF
> From:$USER
> To:$BUGADDAR
> Subject:[Bug Report]
> Configuration Information:
> Machine:$MACHINE
> OS: $OS
> Compiler: $CC
> compilation CFLAG: $CFLAG
> ...
> EOF
● gayat@GSP-HP:~$ cat bugsreport.txt
From:Gayathri
To:bugs@example.com
Subject:[Bug Report]
Configuration Information:
Machine:HP-GSP
OS: Ubuntu 22.04
Compiler: gcc
compilation CFLAG:
```

TEXT EXAMPLE

FILE DESCRIPTORS:

A file descriptor is just a number used by the OS to identify the input and output files.

A **file descriptor n** is a non-negative integer that the operating system uses to **identify and manage open files or input/output resources** for a process.

File Descriptor	Meaning	Default Stream
0	Standard Input	Usually keyboard input
1	Standard Output	Usually the terminal
2	Standard Error	Usually the terminal

So:

- $n=0 \rightarrow \text{stdin}$
- $n=1 \rightarrow \text{stdout}$
- $n=2 \rightarrow \text{stderr}$

In the redirection syntax `n>> file`, you replace **n** with a **file descriptor number**

When a process starts, it usually has three file descriptors open. These correspond to the three standards: standard input (file descriptor 0), standard output (1), and standard error (2). If a process opens additional files for input or output, they are assigned to the next available file descriptors, starting with 3.

`1>` is the same as `>` (Redirecting output), and `0<` is the same as `<` (redirecting input).

Syntax	Meaning
<code>n> file</code>	Redirect FD n output to file
<code>n>> file</code>	Append/write at the end FD n output to file
<code>n>&m</code>	Redirect FD n to FD m () [Redirect the output of file descriptor n to go wherever file descriptor m is already going. So, if m is for Stderr, and n for output, then both output and stderr will go to the same file. Output as the
<code>n< file</code>	Redirect FD n input from file
<code>n<&m</code>	Redirect FD n to get input from FD m
<code>n>&- or <&-</code>	Close FD n

Redirection Type	Valid Syntax	Explanation
Output → Output	<code>n>&m</code>	Redirect output FD n to m
Input → Input	<code>n<&m</code>	Redirect input FD n to m
✗ Mixed (Output/Input)	<code>0>&2 or 2<&0</code>	⚠ Not valid, directions don't match

```
gayat@GSP-HP:~$ echo "Hello STDOUT" 1>output.txt
gayat@GSP-HP:~$ echo "Hello Stderr" 2>errors.txt
Hello Stderr
gayat@GSP-HP:~$ cat out
output.bed output.txt
gayat@GSP-HP:~$ cat output.txt
Hello STDOUT
gayat@GSP-HP:~$ cat errors.txt
gayat@GSP-HP:~$
```

Here, by the first command, you redirected the "Hello STDOUT" to the file output.txt instead of printing to the file while for the 2nd command, it print into the screen and the file remains empty, as echo doesn't produce error.

exec command to open a new file descriptor)

```

● gayat@GSP-HP:~/SummerInternship$ exec 3> file.txt
● gayat@GSP-HP:~/SummerInternship$ echo "Hi this is trial!" >&3
● gayat@GSP-HP:~/SummerInternship$ exec 3>&-
  gayat@GSP-HP:~/SummerInternship$ cat f
  file.txt  file1.txt  file2.txt  file3.txt  file4.txt  file5.txt  fw.sh
● gayat@GSP-HP:~/SummerInternship$ cat file.txt
  Hi this is trial!
○ gayat@GSP-HP:~/SummerInternship$ █

```

Recovery tip:

If you ever close stdout accidentally like you did (exec >&-), you can recover by opening a **new terminal tab/window** — since the current one can't display output anymore.

STRING I/O:

echo, printf, read:

echo:

1. to print standard output
2. syntax: echo -options "what to print"

```

● gayat@GSP-HP:~/SummerInternship$ name=Gayathri
● gayat@GSP-HP:~/SummerInternship$ echo "My name is $name."
  My name is Gayathri.
● gayat@GSP-HP:~/SummerInternship$ █

```

Option	Meaning
-e	Enable backslash escapes
-n	No trailing newline(Starts from the beginning of the line)
-E	explicitly disables interpretation of escape characters (like \n, \t, etc.).

```

● gayat@GSP-HP:~/SummerInternship$ echo "Line1\nLine2"
  Line1\nLine2
● gayat@GSP-HP:~/SummerInternship$ echo -e "Line1\nLine2"
  Line1
  Line2
● gayat@GSP-HP:~/SummerInternship$ echo -n "Progressing..."
  Progressing...
○ Progressing...gayat@GSP-HP:~/SummerInternship$ █

```

```

● Progressing...gayat@GSP-HP:~/SummerInternship$ echo -E "Line1\nLine2"
  Line1\nLine2

```

Escape sequences for echo: used within the text while using options like -e, -n, -E etc.

Escape	Description	Example Output
\n	New line	Hello\nWorld → two lines
\t	Horizontal tab	One\tTwo → tab space

Escape	Description	Example Output
\\	Backslash	\\ → \
\"	Double quote	\ "hello\" → "hello"
\a	Alert (beep sound, if enabled)	 Makes a beep
\b	Backspace (deletes previous char)	ab\bcb → ac
\r	Carriage return (return to line start)	123\rabc → abc
\v	Vertical tab (rarely used)	Moves down & aligns text
\f	Form feed (new page — legacy)	Used in old printers

```

bye
● gayat@GSP-HP:~/SummerInternship$ echo -e "Line1\nLine2"
echo -e "Name:\tGayathri"
echo -e "This is a quote: \"awesome!\""
echo -e "Alerting you\a"
echo -e "12345\b6"      # Deletes 5 → becomes 12346
echo -e "Line with \rOverwrite"
Line1
Line2
Name:   Gayathri
bash: !\: event not found
Alerting you
12346
Overwrite

```

```

● gayat@GSP-HP:~/SummerInternship$ echo -e "1234567\b8"
1234568

```

printf: Used when the data needs to be formatted and then printed to the screen.

printf is used to **format and print** data in a controlled way. It's the calm cousin of echo.

Syntax is `printf format [argument]` where the format decides how the output looks like.

Also, unlike echo, printf won't take you next line after it's work is done

```

● gayat@GSP-HP:~/SummerInternship$ printf "hello world!"
○ hello world!gayat@GSP-HP:~/SummerInternship$

```

For that, use printf "Hello world\n"

```

● gayat@GSP-HP:~/SummerInternship$ printf "hello world\n"
hello world

```

A format specification is preceded by a percent sign (%), and the specifier is one of the characters described below. Two of the main format specifiers are %s for strings and %d for decimal integers.

Format	Description	Example
%s	String	printf "%s" "hi"
%d	Decimal (integer)	printf "%d" 10
%f	Floating point	printf "%f" 3.14
%x	Hex	printf "%x" 255 → ff Unsigned hexadecimal number ; uses a-f for 10 to 15
\n	Newline	printf "Hello\n"
\t	Tab	printf "A\tB"

```

● gayat@GSP-HP:~/SummerInternship$ printf %s "hello"
hello
● gayat@GSP-HP:~/SummerInternship$ printf %d "345"
345
● gayat@GSP-HP:~/SummerInternship$ printf %f "3490087785"
3490087785.000000
● gayat@GSP-HP:~/SummerInternship$ printf %x "3490087785"
d0068369
● gayat@GSP-HP:~/SummerInternship$ printf %x "3490087785\n"
bash: printf: 3490087785\n: invalid number
● gayat@GSP-HP:~/SummerInternship$ printf "3490087785\n"
3490087785
● gayat@GSP-HP:~/SummerInternship$ printf "3490087785\t"
3490087785
○ gayat@GSP-HP:~/SummerInternship$

```

```

● gayat@GSP-HP:~/SummerInternship$ printf "Value:%d\n" 450
Value:450
● gayat@GSP-HP:~/SummerInternship$ printf "Value:450"
Value:450
○ gayat@GSP-HP:~/SummerInternship$
● gayat@GSP-HP:~/SummerInternship$ printf "Price:%f\n" 12.3456
Price:12.345600
● gayat@GSP-HP:~/SummerInternship$ printf "Price:%.3f\n" 12.3456
Price:12.346
● gayat@GSP-HP:~/SummerInternship$ printf "Price:%.1f\n" 12.3456
Price:12.3
● gayat@GSP-HP:~/SummerInternship$ printf "%-5s %5d\n" "Apples" 50
Apples    50
● gayat@GSP-HP:~/SummerInternship$ printf "%s\n is %d\n years old. \n" "Gayathri" 21
Gayathri
is 21
years old.
● gayat@GSP-HP:~/SummerInternship$ printf "%s is %d years old. \n" "Gayathri" 21
Gayathri is 21 years old.

```

In the example, %-5s %5d\n, %-5s is left align with width 5, %5s, right align with 5 width

Precision and flags in printf:

For floating numbers:

Format	Output
%.2f	2 decimal places
%.0f	Rounds to integer
%.4f	4 decimal places

Field alignment

Format	Meaning
%5d	Right-align number in width 5
%-5s	Left-align string in width 5

Format	Meaning
%05d	Pad number with zeros (width 5)

Flags:

Flag	Effect
-	Left-align within width
0	Zero-padding (only for numbers)
.	Start of precision (for floats/strings)

```
gayat@GSP-HP:~/SummerInternship$ printf "%-05s hehe"
hehegayat@GSP-HP:~/SummerInternship$ printf "%-5s hehe"
hehegayat@GSP-HP:~/SummerInternship$
```

Truncating strings:

```
gayat@GSP-HP:~/SummerInternship$ printf "%.4s\n" "Gayathri"
Gaya
```

Read: used to take input from the user and keep it in a variable

Syntax: *read variable*

```
gayat@GSP-HP:~/SummerInternship$ echo "What is your name:"; read name; echo "hello, $name"
What is your name:
GSP
hello, GSP
```

To read multiple input:

```
gayat@GSP-HP:~/SummerInternship$ read first middle last
gayathri s pramod
gayat@GSP-HP:~/SummerInternship$ echo $first
gayathri
gayat@GSP-HP:~/SummerInternship$ echo $middle
s
gayat@GSP-HP:~/SummerInternship$ echo $last
pramod
```


Option	Meaning
-p	Prompt user without using echo
-s	Silent (useful for passwords)--
-t	Timeout (auto skip after seconds)
-n	Read limited number of characters

```

password.-1234: Command not found
● gayat@GSP-HP:~/SummerInternship$ password=1234
⊗ gayat@GSP-HP:~/SummerInternship$ read "Password"
● gayat@GSP-HP:~/SummerInternship$ read -s "Password"
○ gayat@GSP-HP:~/SummerInternship$

```

here the password,

when typed in, will not be seen.

COMMAND BLOCKS:

Used to group commands together

```

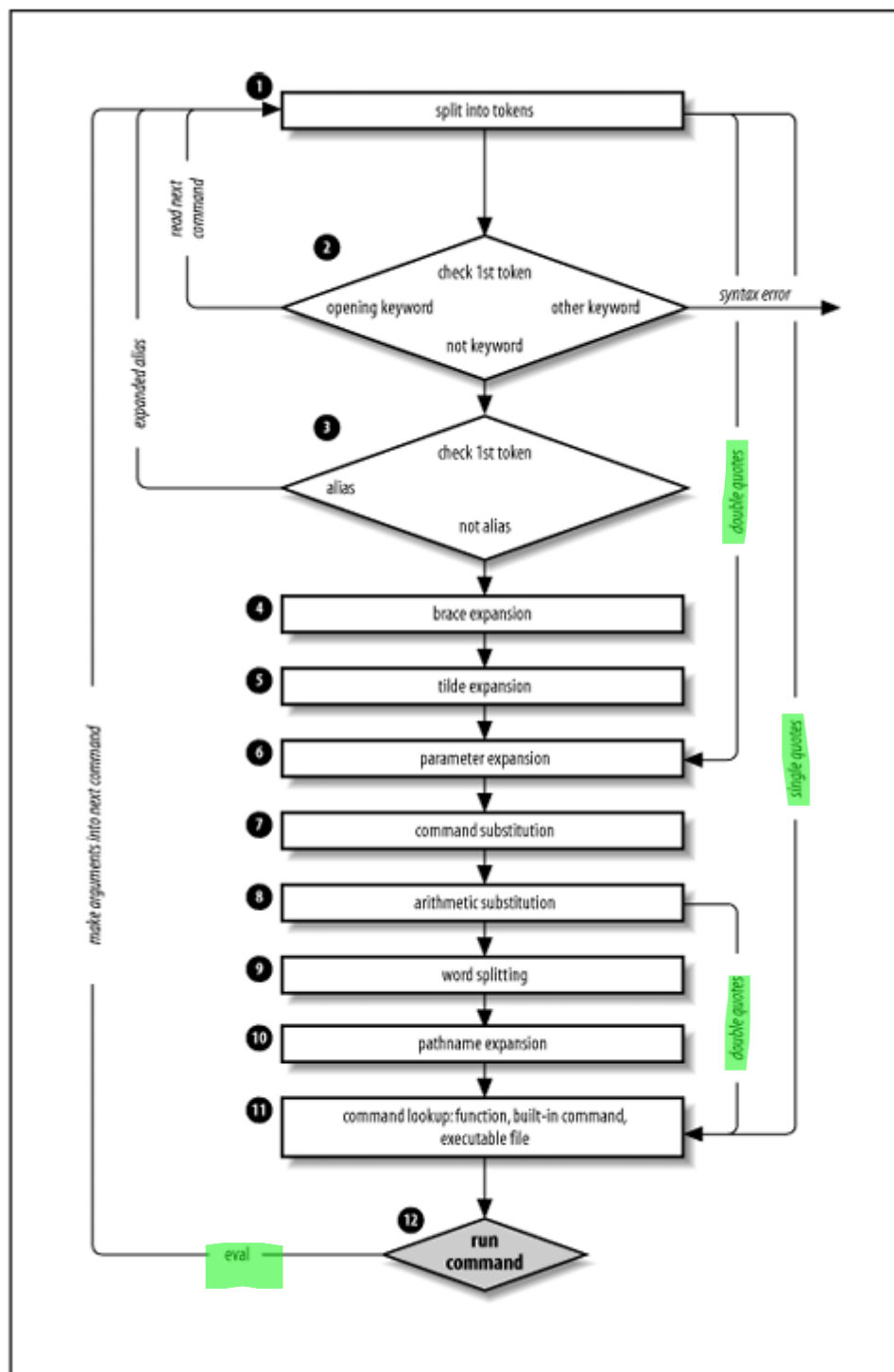
{ cmd1; cmd2; } > out.txt           ( cd dir && ls )

Runs both, and redirects output together.   Runs in a subshell, so changes (like cd) don't affect your
Don't forget the ; and space before }.      current shell.

```

{} = group in current shell; **()** = run in subshell.

Command Line Processing:



Check the double and single quotes

Five important ways to modify this process:

quoting;

using command, builtin, or enable;

using the advanced command eval.

Quotations:

" → removes all special meaning--No variable or command substitution — **literal text only**

"" → removes some of the special meanings--Escapes special characters **except** \$, `, and \

` → runs a command and substitutes its output

Quote Type	Allows Variable Expansion	Allows Command Substitution	Escapes Special Characters
" "	✓ Yes	✓ Yes	✗ Not fully
' '	✗ No	✗ No	✓ Everything is literal
` ` / \$(...)	✓ Yes	✓ Yes	✓ Output inserted

Expression	Value
\$person	hatter
"\$person"	hatter
\\$person	\$person
`\$person`	\$person

Expression	Value
""\$person""	'hatter'
~alice	/home/alice
"~alice"	~alice
`~alice`	~alice

```
• gayat@GSP-HP:~/SummerInternship$ name=Gayathri
• gayat@GSP-HP:~/SummerInternship$ echo "1.$name"
1.Gayathri
• gayat@GSP-HP:~/SummerInternship$ echo '1.$name'
1.$name
• gayat@GSP-HP:~/SummerInternship$ echo `1.$name`
1.Gayathri: command not found
```

Command, builtin and enable:

Command → runs a command without shell function overrides—when there exists a function and an external command of the same name.

Builtin → runs a builtin command even if a function of same name exists; same use as command but exclusively for built-in keywords

Enable → enable or disable builtin and also to list all builtins

```
enable          # Lists all builtins
enable -n echo  # Disables echo
echo "Hi"       # Will now fail!
enable echo     # Re-enables it
```

Eval command:

Evaluation of a string to run it as a command.

```
eval "command string"
```

The syntax of eval:

Use Case	Example
Evaluate dynamic variable	eval "echo \\$\$var"
Run dynamic command	eval "\$cmd"
Dangerous with raw user input ⚠	Never directly use input